MCP-Lock: Provenance and Tool-Definition Pinning to Stop AI Coding Agents from Silently Running Unreviewed Tool Servers

Domain: Cybersecurity · Software Supply-Chain Security · AI Agents (Model Context Protocol)

Guide: Mr. Sai Prasanth D

Assistant Professor Jr G

Department of Data Science and Business Systems

Aakash G S (RA2311042010060)

4th Year, B.Tech, CSBS, DSBS Department

Hariharan J (RA2311042010059)

4th Year, B.Tech, CSBS, DSBS Department

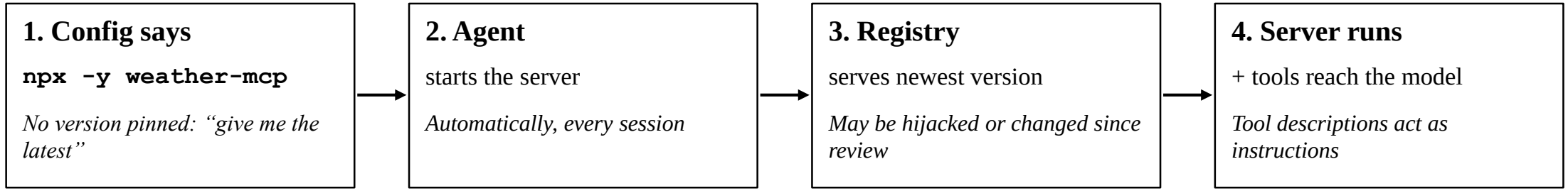
J Jaswanth Raj (RA2311042010013)

4th Year, B.Tech, CSBS, DSBS Department

Problem Statement & Background

Background

- AI coding agents (Claude Code, Cursor, …) get their tools from MCP servers, which they start straight from npm or PyPI with unpinned commands such as npx -y.



Problem Statement

- Unpinned MCP-server launches let the code and the tool descriptions an agent runs change after a human reviewed them, with no signal to the user; existing pre-install checks look only at package names, sources and age. Develop a gate that, before every launch and without executing package code, detects when the version about to run departs from the reviewed one in build provenance, publisher, install scripts or tool definitions, and that catches more real attacks than existing checks at a measured false-block cost.

MCP = Model Context Protocol, the open standard through which agents call external tools. Rug pull: a server changes a tool after it was approved.

Motivation & Justification

9.8%	**500+**	**36.5%**	**Rarely**
of 2,660 real agent setups launch MCP servers unpinned [1]	npm packages hit by the Shai-Hulud worm through stolen maintainer tokens, Sept. 2025 [2]	of 2,000 sampled MCP packages changed their tools within 90 days (our pilot census)	do coding assistants check trust signals before installing anything [3]

- **Why now:** agents start tool servers unattended, worms already used stolen npm tokens, and tool definitions change often without any metadata signal.
- **Target beneficiaries:** developers using AI coding agents (their keys, files and tokens stay safe); companies and security teams (one auditable lockfile for every approved tool server); open-source maintainers and registries (a hijacked release no longer spreads silently).
- **Scope (in):** MCP servers launched from npm and PyPI; attackers who squat a name, steal a token, enter the release pipeline, own the package, or own a dependency.
- **Scope (out):** compromise of the registry itself, forged Sigstore signatures, attacks on the language model.

The 36.5% figure is from our RQ1 census (stratified random sample, tool definitions extracted statically). Numbers in [ ] refer to the References slide.

Literature Survey

Paper / Authors	Year	Method / Contribution	Limitation
Ladisa et al. [4]	2023	IEEE S&P: SoK taxonomy of attacks on open-source supply chains, incl. account takeover and install scripts	General package ecosystems; nothing on agents, MCP or tool descriptions
Hou et al. [5]	2025	Survey of the MCP landscape; defines tool poisoning and rug-pull threats	Survey only: no defence measured on real packages
Bhatt et al. [6]	2025	ETDI: OAuth-signed, versioned tool definitions with policy-based access control	Needs servers and registries to change and sign; not usable with existing unsigned packages
Bagmar & Saraf [7]	2026	Pre-install hook for coding agents: 7 checks on name, source and age (pip)	Never compares with the version you reviewed; misses hijacks of existing packages
Kapner et al. [1]	2026	Scans 2,660 coding-agent setups; finds 9.8% with unpinned MCP servers	Measures exposure only; no provenance analysis and no defence
Kraishan [8]	2026	Census of drift in MCP-registry fields over 8,900 multi-version servers	Registry does not version tool definitions, so tool changes go unmeasured; recommends client-side pinning

Common thread: no work checks, before launch, whether a server's build, publisher, install scripts and tools still match what was reviewed.

All six sources verified against their primary pages (docs/CITATIONS_VERIFIED.md). Ladisa et al., IEEE S&P 2023, DOI 10.1109/SP46215.2023.10179304.

Literature Survey (contd.)

Paper / Authors	Year	Method / Contribution	Limitation
Sejfia & Schäfer [9]	2022	ICSE: Amalfi, machine-learning detection of malicious npm packages using three complementary techniques; 95 unknown samples found in one week	Flags malicious new packages; keeps no record of the version a user reviewed and ignores tool definitions
Ladisa et al. [10]	2023	ACSAC: language-independent features to detect malicious npm and PyPI packages; 58 unknown ones found in 31,292	Classification of package code; no continuity check and nothing specific to agents
Schorlemmer et al. [11]	2024	IEEE S&P: measures signing in Maven, PyPI, DockerHub and Hugging Face; registry policy drives how much is signed	Measurement only; many packages carry no signature, so a defence cannot rely on signatures alone
Gamage et al. [12]	2025	Compares lockfile design across seven package managers, with 15 developer interviews	Covers dependency lockfiles; nothing on MCP tool definitions or build-identity continuity
Gaire et al. [13]	2025	SoK: taxonomy of security and safety risks in the MCP ecosystem and a review of defences	Systematization; no new defence evaluated on real attacks
Huang et al. [14]	2026	STRIDE/DREAD threat model of MCP; tests tool poisoning on seven MCP clients and proposes layered defences	Client-side view; does not cover package versions, install scripts or registry provenance

Common thread: detection and signing work looks for known-bad code or a signature; none checks an agent's tool server against the state that was reviewed.

Checked on the publisher or arXiv page of each paper, 4 Nov. 2026.

Research Gap & Market / Patent Study

Existing solution	Before launch, runs no code	Pins tool definitions	Build, publisher, script continuity	No server / registry change
Bagmar & Saraf hook [7]	✓	✗	✗	✓
MCP-Scan, mcp-pin, mcpseal [15]–[17]	✗	✓	partly	✓
Signed definitions: ETDI [6]	–	✓	–	✗
npm provenance, Sigstore [18], [19]	✓	✗	partly	✓
MCP-Lock (proposed)	✓	✓	✓	✓

Research gaps

- **G1:** no measurement of how often MCP tool definitions really change.
- **G2:** no check reads tool definitions before launch, without starting the server.
- **G3:** no continuity checks on build, publisher and install scripts per server.
- **G4:** no evaluation on real attacks with the false-block cost on honest upgrades.

Patent search: Google Patents, 3 Nov. 2026, not exhaustive.

Patent landscape

- **US 12,314,394 B2 (Socket, 2025) [20]:** scans a project's dependency files for malware via a cache of scanned packages; no comparison with a reviewed version.
- **US 10,055,576 B2 (Red Hat, 2018) [21]:** blocks repository packages whose components are on an insecure list; known-bad lists only.
- *No patent found that pins agent tool definitions before launch.*

Market and our edge

- Socket and npm provenance protect packages; MCP-Scan and the pinning proxies protect tools only after start-up.
- Simple alternatives: exact version pinning (freezes every honest upgrade) and npm --ignore-scripts (stops install scripts only).
- MCP-Lock: one lockfile pins archive, provenance, publisher, install scripts and tool definitions; the gate checks all five before every launch, needs no ecosystem change, and closes G1–G4.

Objectives / Product Goals

Objective	Specific, measurable target	Gap	Status at Review 0
O1. Measure the problem	Census MCP packages on npm and PyPI; extract tool definitions from a stratified sample of 2,000 and report the 90-day change rate with 95% intervals.	G1	Done: 15,272 packages; 36.5% changed tools
O2. Build MCP-Lock	A lockfile and a launch gate with 5 checks (S1–S5) that never runs package code, installable as a coding-agent hook and covered by automated tests.	G2, G3	Done: gate + hook, 103 tests
O3. Prove it works	Pre-registered test on ≥ 490 real attacks: detect more than the baseline (McNemar, $\alpha = 0.05$); report false blocks on 8,180 honest upgrades; withdraw the claim if S2–S5 add < 2.	G4	Done: 39.8% vs 19.6%, +0.42 pp
O4. Stress-test and publish	Build 12 adaptive attacks, add hardening with measured costs, release code and data, and submit an IEEE paper before the final review.	G2–G4	Planned: paper drafted, to submit

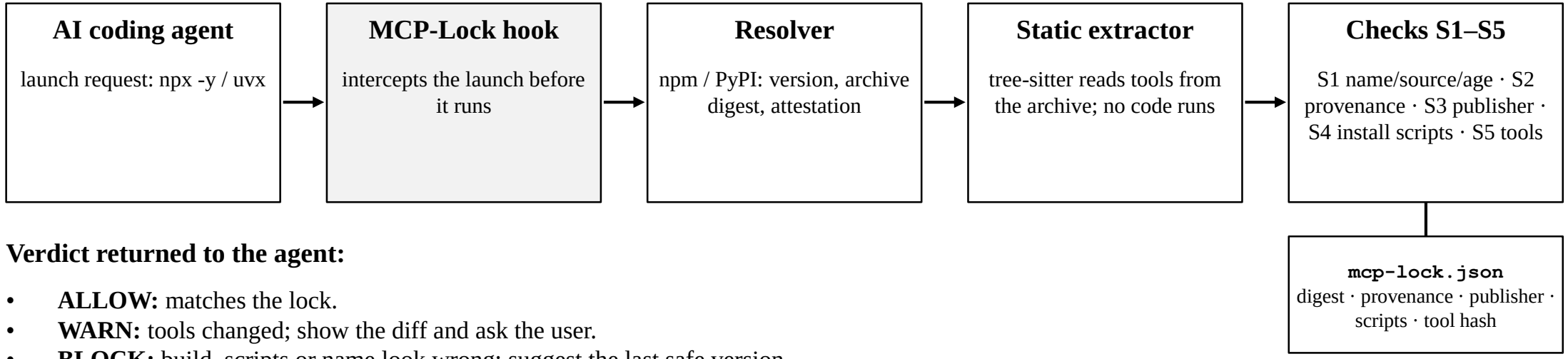
SMART: S specific task · M a number or test · A sized to a 3-person team · R tied to gap G1–G4 · T each closes at a review milestone (see work plan).

Alignment with UN Sustainable Development Goals

Goal	Target	How MCP-Lock contributes
SDG 9: Industry, Innovation and Infrastructure	**9.1**	Reliable, resilient infrastructure: the open-source supply chain is digital infrastructure; MCP-Lock keeps one hijacked release from reaching every agent that launches it.
	9.5	Upgrade technological capability: a new, pre-registered security method with public code.
SDG 16: Peace, Justice and Strong Institutions	**16.a**	Capacity to prevent crime: blocks the cyber-crime pattern behind worms like Shai-Hulud (stolen tokens, install-script payloads).
	16.6	Accountable, transparent systems: every verdict is traceable to an auditable lockfile.
SDG 17: Partnerships for the Goals	**17.6**	Knowledge sharing in science and technology: code, data, replay bundles and the OSF pre-registration are open, so others, including developing-country teams, can reuse and verify them.

Targets as defined in UN resolution A/RES/70/1, "Transforming our world: the 2030 Agenda for Sustainable Development" [23].

Proposed Methodology / High-Level Architecture



Verdict returned to the agent:

- **ALLOW:** matches the lock.
- **WARN:** tools changed; show the diff and ask the user.
- **BLOCK:** build, scripts or name look wrong; suggest the last safe version.

Modules

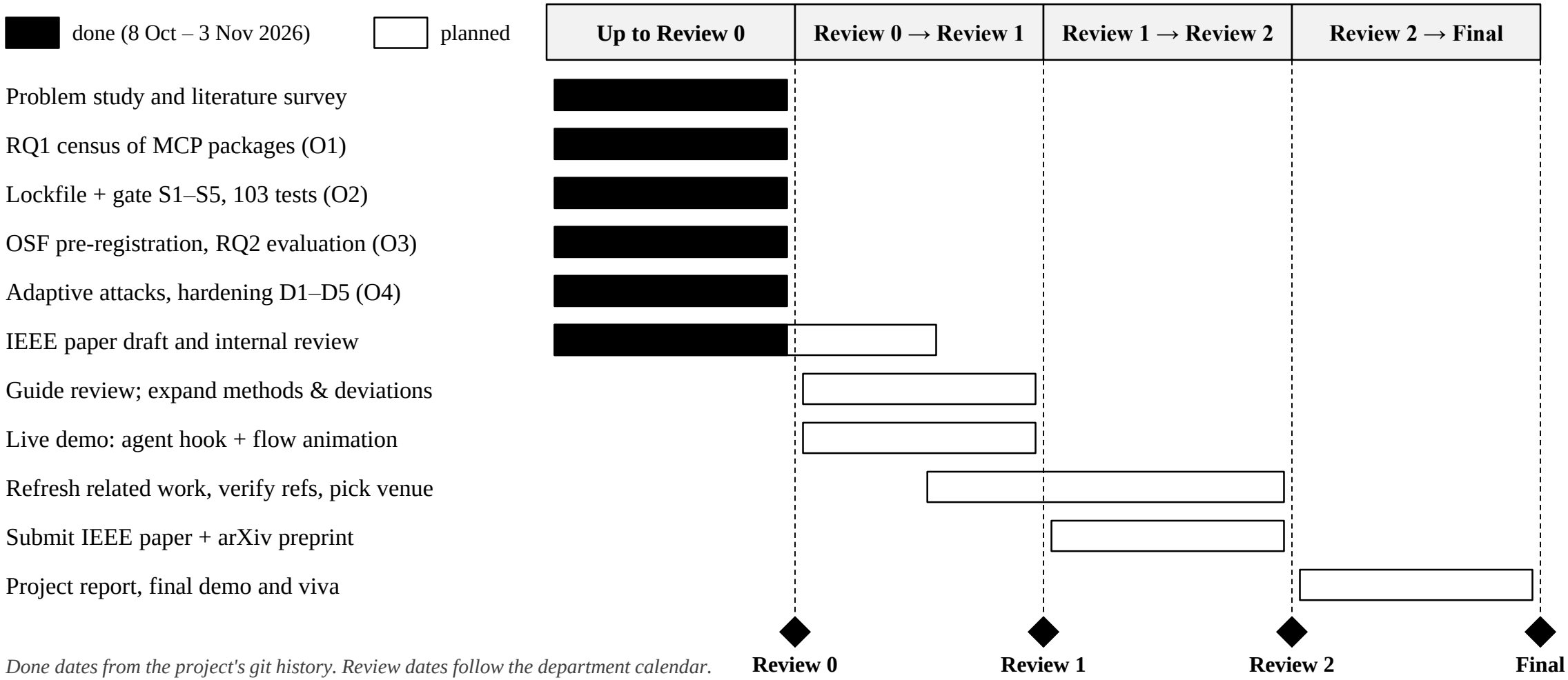
- **census/** RQ1: frame of MCP packages, version history, static tool extraction
- **lockfile/** lock format spec and canonical tool-definition hashing
- **gate/** checks S1–S5, verdicts, safe-version suggestions, agent hook, CLI
- **eval/** replay of 490 attacks and 8,180 upgrades; adaptive attacks in Docker

Technology Stack

Python 3.11, tree-sitter (JS/TS), npm & PyPI APIs, Sigstore / SLSA provenance, Docker sandbox, pytest + GitHub Actions, Claude Code hook, OSF pre-registration, LaTeX IEEEtran.

Method: lock once after review (mcplock lock), then the gate checks every launch. Evaluation is pre-registered on OSF [22] against the baseline hook [7].

Work Plan & Review Timeline



Done dates from the project's git history. Review dates follow the department calendar.

References

[1] B. Kapner, C. Soceanu, A. Petrunin, and H. Gartner, “Scanning the harness: Configuration exposures in AI coding-agent supply chains,” arXiv:2609.07360v3, 2026.

[2] CISA, “Widespread supply chain compromise impacting npm ecosystem,” Alert, Sept. 23, 2025.

[3] P. Shan, “Do AI coding assistants check before they install? A pre-registered demand-side audit of trust signals in the research software supply chain,” arXiv:2609.07754, 2026.

[4] P. Ladisa, H. Plate, M. Martinez, and O. Barais, “SoK: Taxonomy of attacks on open-source software supply chains,” in Proc. IEEE Symp. Security and Privacy (SP), 2023, pp. 1509–1526, doi: 10.1109/SP46215.2023.10179304.

[5] X. Hou, Y. Zhao, S. Wang, and H. Wang, “Model Context Protocol (MCP): Landscape, security threats, and future research directions,” arXiv:2503.23278, 2025.

[6] M. Bhatt, V. S. Narajala, and I. Habler, “ETDI: Mitigating tool squatting and rug pull attacks in Model Context Protocol (MCP) by using OAuth-enhanced tool definitions and policy-based access control,” arXiv:2506.01333, 2025.

[7] A. Bagmar and P. Saraf, “Setup complete, now you are compromised: Weaponizing setup instructions against AI coding agents,” arXiv:2607.15143, 2026.

[8] O. Kraishan, “Same name, different server: A security census of silent drift in the Model Context Protocol ecosystem,” arXiv:2609.14119, 2026.

[9] A. Sejfia and M. Schäfer, “Practical automated detection of malicious npm packages,” in Proc. 44th Int. Conf. Software Engineering (ICSE), 2022, doi: 10.1145/3510003.3510104.

[10] P. Ladisa, S. E. Ponta, N. Ronzoni, M. Martinez, and O. Barais, “On the feasibility of cross-language detection of malicious packages in npm and PyPI,” in Proc. ACSAC, 2023, pp. 71–82, doi: 10.1145/3627106.3627138.

[11] T. R. Schorlemmer et al., “Signing in four public software package registries: Quantity, quality, and influencing factors,” in Proc. IEEE Symp. Security and Privacy (SP), 2024, doi: 10.1109/SP54263.2024.00182.

[12] Y. Gamage, D. Tiwari, M. Monperrus, and B. Baudry, “The design space of lockfiles across package managers,” arXiv:2505.04834, 2025.

[13] S. Gaire, S. Gyawali, S. Mishra, S. Niroula, D. Thakur, and U. Yadav, “Systematization of knowledge: Security and safety in the Model Context Protocol ecosystem,” arXiv:2512.08290, 2025.

[14] C. Huang, X. Huang, N. P. Tran, and A. Milani Fard, “Model Context Protocol threat modeling and analyzing vulnerabilities to prompt injection with tool poisoning,” arXiv:2603.22489, 2026.

[15] L. Beurer-Kellner and M. Fischer, “Introducing MCP-Scan: Protecting MCP with Invariant,” Invariant Labs blog, Apr. 2025.

[16] GautamTalksDev, “mcp-pin: Pin what your agent approved; block it when it changes,” GitHub, 2026. [Online]. Available: github.com/GautamTalksDev/mcp-pin

[17] confuseddude, “mcpseal: Pins the hash of every approved MCP tool definition,” GitHub, 2026. [Online]. Available: github.com/confuseddude/mcpseal

[18] npm, “Generating provenance statements,” npm Docs, 2026. [Online]. Available: docs.npmjs.com/generating-provenance-statements

[19] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software signing for everybody,” in Proc. ACM CCS, 2022, pp. 2353–2367, doi: 10.1145/3548606.3560596.

[20] F. H. Aboukhadijeh and M. C. Lysenko, “Detecting suspected malware in open-source dependencies,” U.S. Patent 12,314,394 B2, May 27, 2025.

[21] S. B. Milner and J. R. Bowes, “Detection of malicious software packages,” U.S. Patent 10,055,576 B2, Aug. 21, 2018.

[22] A. G S, “MCP-Lock: Does provenance and tool-definition pinning catch MCP-server supply-chain attacks that name/source checks miss, and at what false-block cost?” OSF Registries, Oct. 2026, doi: 10.17605/OSF.IO/2AMJ3.

[23] United Nations General Assembly, “Transforming our world: The 2030 Agenda for Sustainable Development,” Resolution A/RES/70/1, Oct. 2015.

Code and data: github.com/ftaakash/MCP-Lock. All entries checked against their primary source; patents checked on Google Patents.